Date
Page

OPerating Systems

Unit wise Solution:

Past year Solutions:

Name - Suman Kushwaha
Roll = 03
Subject = OPerating Systems
Semester = 4th
github = Kushwaha-Suman
website = KushwahaSuman.com.np

2083-03-01

Date
Page

Unit - 1

Operating System Overview

(1) Write Short notes on:
a) Linux Scheduling (2080)
b) Fragmentation

a) Linux Scheduling:

⇒ Linux Scheduling is the process by which the linux operating system decides which process gets access to the CPU and for how long. It is responsible for managing multiple process and ensuring efficient CPU utilization.

Objectives of linux Scheduling:

- fair allocation of CPU time among processes.
- maximum CPU utilization.
- fast response time for interactive processes
- Efficient execution of processes.

features:

- Supports preemptive multitasking, where a running process can be interrupted.
- Uses process' priorities to determine execution order
- Employs scheduling algorithms such as the completely fair Scheduler (CFS)

Date
Page

Conclusion:

→ linux scheduling improves system performance by ensuring that all processes get a fair chance to execute while maintaining efficient CPU usage.

b) Fragmentation

→ Fragmentation is the condition in which memory is wasted because memory is allocated inefficiently. It reduces memory utilization and can affect system performance.

Types of fragmentation:

1 Internal fragmentation:

→ occurs when a process is allocated more memory than it requires. The unused space inside the allocated memory block is wasted.

e.g If a process needs 18 Kb and is allocated 20 Kb, the remaining 2 Kb is wasted.

Date
Page

2 External fragmentation

→ occurs when free memory is divided into many small non-contiguous blocks. Although the total free memory may be sufficient, it cannot be used because it is not contiguous.

Eg free memory block of 10 KB, 5 KB, and 15 KB exist separately, but a process requiring 25 KB cannot be allocated memory

Conclusion:

→ fragmentation leads to inefficient memory usage, technique such as compaction and paging can help reduce fragmentation

(2) Define the terms shell and system call. How it is handled? Illustrate with suitable example. (2080)

1 Shell:

→ A Shell is a command interpreter that acts as interface between the user and the operating system. It accepts commands from the user and passes them to the operating system for execution.

eg Bash, sh, Csh, in linux

2. System Call:

→ A system call is a mechanism through which a user program requests to services from the operating system Kernel such as file operations, process managment and memory allocation.

e.g open(), read(), write(), fork(), close()

How is a system call Handled?

1. The user enters a command in the shell.
2. The shell starts the corresponding program.
3. The program requests an OS service using a system call.
4. control is transferred from user mode to kernel mode
5. The kernel performs the requested operation.
6. control returns to the user program, and the result is displayed.

Date
Page

e.g Suppose the user enters
cat file.txt

Handling Process:

- The shell receives the command cat file.txt.
- The shell execute the cat program.
- The cat program uses system call such as open(), read(), and close() to access the file.
- The kernel reads the file contents and returns them to the program.
- The contents are displayed on the screen

Conclusion:

→ The shell provides the interface between the user and the operating system, while system calls provide the interface between user programs and the kernel. together they enable users and applications to access operating system servies efficiently

Date
Page

③ What is system call? Discuss ~~the~~ process of handling system calls briefly (2078)

→ System call:

A system call is a mechanism that allows a user program to request services from the operating system kernel. It provides an interface between user programs and the operating system.

e.g open(), read(), write(), fork(), close()

Process of Handling System calls:

1. The users program requests a service (e.g file access)
2. A system call is invokes in the program.
3. The CPU switches from user mode to kernel mode.
4. The operating system kernel identifies the requested system call.
5. The kernel performs the required operation.
6. The result is returned to the user program.
7. Control switches back to user mode.

eg When a program wants to read a file, it uses a the read() system call. the kernel accesses the file from storage and returns the data to the program